CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Database Management Systems & System Architecture: Placement Master Edition (Prescribed Textbook)

Subject Track: DBMS & Database Design

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Authoritative campus placement textbook covering Database Management Systems architecture, Entity-Relationship modeling, Relational Algebra, Normalization (1NF through 5NF), B+ Trees, 2PL, ACID transaction management, CAP theorem, and System Design.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for DBMS & Database Design on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (8 CHAPTERS):

Ch 1: Database Architecture & Three-Schema Tier Architecture

Ch 2: Entity-Relationship (ER) Modeling & Schema Mapping

Ch 3: Relational Algebra & Relational Calculus

Ch 4: Functional Dependencies & Higher Normal Forms

Ch 5: Storage Structures, File Organization & Indexing

Ch 6: Transaction Processing, Conflict Serializability & Strict 2PL

Ch 7: Distributed Databases, Replication & CAP Theorem

Ch 8: High-Level System Design for Campus Placements

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: DBMS & Database Design

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Authoritative campus placement textbook covering Database Management Systems architecture, Entity-Relationship modeling, Relational Algebra, Normalization (1NF through 5NF), B+ Trees, 2PL, ACID transaction management, CAP theorem, and System Design.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: Database Architecture & Three-Schema Tier Architecture](#chapter-1)
- [Chapter 2: Entity-Relationship (ER) Modeling & Schema Mapping](#chapter-2)
- [Chapter 3: Relational Algebra & Relational Calculus](#chapter-3)
- [Chapter 4: Functional Dependencies & Higher Normal Forms](#chapter-4)
- [Chapter 5: Storage Structures, File Organization & Indexing](#chapter-5)
- [Chapter 6: Transaction Processing, Conflict Serializability & Strict 2PL](#chapter-6)
- [Chapter 7: Distributed Databases, Replication & CAP Theorem](#chapter-7)
- [Chapter 8: High-Level System Design for Campus Placements](#chapter-8)

CHAPTER 1

Database Architecture & Three-Schema Tier Architecture

Key Principle / Exam Takeaway: Master Physical vs Logical Data Independence, 2-tier vs 3-tier architectures, DDL vs DML, and DBMS component modules.

Core Curriculum Topics Covered in this Chapter:

- Data models: Relational, Hierarchical, Network, Document, Graph
- ANSI/SPARC Three-Schema Architecture: Physical, Logical/Conceptual, and View/External levels
- Physical Data Independence vs Logical Data Independence
- Database engine subsystems: Storage Manager, Buffer Pool Manager, Query Optimizer, Transaction Manager
- Data Definition Language (DDL) vs Data Manipulation Language (DML) vs DCL and TCL

· 1. Core Concepts & In-Depth Explanation

A Database Management System (DBMS) is specialized system software designed to store, manage, and retrieve structured data reliably, efficiently, and concurrently.

ANSI/SPARC Three-Schema Architecture: The hallmark of relational databases is separating user applications from underlying physical storage through three abstraction tiers: 1. **External Level (Views):** How individual end-users or application modules perceive the data. Hides irrelevant schema details and enforces row/column security. 2. **Conceptual Level (Logical Schema):** The complete global logical design of the entire database: all entities, attributes, relationships, data types, and enterprise integrity constraints. 3. **Internal Level (Physical Schema):** How bytes are physically organized on disk storage: file structures, B+ Tree index allocations, block sizes, and compression algorithms.

Data Independence:

- **Physical Data Independence:** The capacity to modify physical storage structures (e.g., creating a new index or switching from HDD to NVMe SSD) without altering the logical schema or user application code.
- **Logical Data Independence:** The capacity to modify the conceptual schema (e.g., adding an optional column or splitting a table into two normalized relations) without breaking existing external user views or requiring rewrites of existing queries.

· 2. Key Formulas, Rules & Complexity Classes

1. **Three-Schema Abstraction Layers:** $$\text{External Views} \iff \text{Conceptual Logical Schema} \iff \text{Internal Physical Disk Organization}$$ 2. **Data Independence Axiom:** $$\text{Physical Independence} \gg \text{Easier to achieve than Logical Independence}$$

· 3. Step-by-Step Solved Examples

Example 1.1: Demonstrating Logical Data Independence with Database Views

- **Difficulty / Target Companies:** `Easy | Oracle, Amazon, TCS`

- **Problem Statement:** A company normalizes table `Employee(emp_id, name, dept_name, salary)` by splitting it into `Emp` and `Dept`. How do we achieve Logical Data Independence so existing reporting tools continue working without modifying their queries?

Step-by-Step Walkthrough & Logic:

- Step 1: Decompose physical tables into `Emp(emp_id, name, dept_id, salary)` and `Dept(dept_id, dept_name)`.
- Step 2: Create a View matching the legacy table name `Employee` that joins `Emp` and `Dept`.
- Step 3: Existing client applications query `SELECT * FROM Employee` without modification.

Complete Solved Code / Calculation:

```
-- Decomposed Conceptual Tables:
CREATE TABLE Dept (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL
);

CREATE TABLE Emp (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    dept_id INT REFERENCES Dept(dept_id),
    salary DECIMAL(10,2)
);

-- External View granting Logical Data Independence:
CREATE VIEW Employee AS
SELECT e.emp_id, e.name, d.dept_name, e.salary
FROM Emp e
JOIN Dept d ON e.dept_id = d.dept_id;
```

- **Line-by-Line Breakdown & Complexity Analysis:** Existing queries execute against the View without realizing that underlying physical schemas were partitioned.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: Which level of the Three-Schema architecture defines the physical file organization and index structures?

Options:

- A) External Level
- B) Conceptual Level
- C) Internal (Physical) Level
- D) User Level

Answer: **C) Internal (Physical) Level** **Detailed Solution & Reasoning:** The Internal schema describes physical storage details, record layouts, and indexing mechanisms.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Amazon)

- **Interview Question:** _What is the Buffer Pool Manager in a DBMS and how does it prevent disk I/O bottlenecks?_

- **Model High-Scoring Answer:** The Buffer Pool Manager manages an in-memory cache of physical database pages in RAM. When a query requests a page, the engine first checks the Buffer Pool. If present (Buffer Cache Hit), access takes nanoseconds with zero disk I/O. If missing (Buffer Miss), the manager allocates a frame, reads the page from disk into RAM, and manages dirty page eviction back to disk using algorithms like LRU-K or Clock.

- **Interviewer Follow-Up & Pro Tip:** ? Mention dirty page flushing and write-ahead logging synchronization.

CHAPTER 2

Entity-Relationship (ER) Modeling & Schema Mapping

Key Principle / Exam Takeaway: Master Entities, Attributes, Relationships, Cardinality ratios (1:1, 1:N, M:N), Weak Entities, and ER-to-Relational schema conversion rules.

Core Curriculum Topics Covered in this Chapter:

- ER model constructs: Entity sets, Weak Entity sets, Identifying relationships
- Attributes: Simple vs Composite, Single-valued vs Multi-valued, Stored vs Derived
- Mapping Cardinalities: One-to-One (1:1), One-to-Many (1:N), Many-to-Many (M:N)
- Participation constraints: Total participation (double line) vs Partial participation
- ER-to-Relational mapping rules: Handling Multi-valued attributes and M:N relationship tables

· 1. Core Concepts & In-Depth Explanation

Entity-Relationship (ER) modeling is a high-level conceptual data modeling framework translating real-world business requirements into visual diagrammatic schemas:

- **Entity:** An identifiable real-world object (e.g., `Student`, `Course`).
- **Weak Entity:** An entity that cannot be uniquely identified by its own attributes alone and relies on an identifying relationship with a strong owner entity. It has a **Partial Key (Discriminator)** underlined with a dashed line.
- **Multi-Valued Attribute:** An attribute that can hold multiple values for a single entity (e.g., `phone_numbers`). In relational mapping, multi-valued attributes **must be split into a separate table** with a foreign key back to the parent entity to satisfy 1NF.
- **Many-to-Many (M:N) Relationship:** Cannot be represented by adding foreign keys to either entity table; it **must be mapped into a distinct junction/associative table** combining the primary keys of both participating entities as a composite primary key.

· 2. Key Formulas, Rules & Complexity Classes

1. **Relational Mapping Rules:**

- **Multi-valued Attribute:** Table $R(\underline{\text{ParentPK}}, \underline{\text{AttributeValue}})$
- **M:N Relationship:** Junction Table $R(\underline{\text{EntityA_PK}}, \underline{\text{EntityB_PK}}, \text{Attributes})$

· 3. Step-by-Step Solved Examples

Example 2.1: Mapping Many-to-Many Relationship to Relational Tables

- **Difficulty / Target Companies:** `Easy | Oracle, Infosys, TCS`
- **Problem Statement:** Map an ER diagram where Students enroll in multiple Courses and Courses contain multiple Students into normalized SQL tables.

Step-by-Step Walkthrough & Logic:

- Step 1: Create strong entity table `Student(student_id, name)`.
- Step 2: Create strong entity table `Course(course_id, title)`.
- Step 3: Create junction table `Enrollment(student_id, course_id, enrollment_date)` with composite primary key `(student_id, course_id)`.

Complete Solved Code / Calculation:

```
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL
);

-- Associative Junction Table for M:N Relationship:
CREATE TABLE Enrollment (
    student_id INT REFERENCES Student(student_id) ON DELETE CASCADE,
    course_id INT REFERENCES Course(course_id) ON DELETE CASCADE,
    enrollment_date DATE NOT NULL,
    PRIMARY KEY (student_id, course_id)
);
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees no redundant duplication while capturing enrollment relationship attributes.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: How is a Weak Entity set identified in a relational database?

Options:

- A) By its own primary key
- B) By combining the primary key of its identifying strong entity with its own partial discriminator
- C) Using null values
- D) With an index only

Answer: **B) By combining the primary key of its identifying strong entity with its own partial discriminator** **Detailed Solution & Reasoning:** A weak entity requires the owner's primary key plus its local discriminator to achieve unique tuple identity.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Microsoft)

- **Interview Question:** _What is the difference between Total and Partial participation in ER modeling?_

- **Model High-Scoring Answer:** Participation constraints specify whether all entities in an entity set must participate in a relationship. In **Total Participation** (represented by double lines in Chen notation), every entity in the set must participate in at least one relationship instance (e.g., every `Employee` MUST belong to a `Department`). In the database, this is enforced via a `NOT NULL` foreign key. In **Partial Participation**, some entities may not participate (e.g., not every `Employee` manages a `Department`), mapped as a nullable foreign key.

- **Interviewer Follow-Up & Pro Tip:** ? Connect total participation directly to NOT NULL foreign key constraints.

CHAPTER 3

Relational Algebra & Relational Calculus

Key Principle / Exam Takeaway: Master procedural Relational Algebra (Select, Project, Union, Set Difference, Cartesian Product, Joins) and non-procedural Tuple Relational Calculus (TRC).

Core Curriculum Topics Covered in this Chapter:

- Relational Algebra fundamental operators: Selection (sigma), Projection (pi), Union (cup), Set Difference (-), Cartesian Product (x)
- Derived operators: Natural Join (bowtie), Theta Join, Equi-Join, Division operator (/)
- Relational Completeness and equivalence between Relational Algebra and TRC/DRC
- Tuple Relational Calculus (TRC) existential and universal quantifiers (exists, forall)
- Query optimization using relational algebra heuristic push-down transformations

· 1. Core Concepts & In-Depth Explanation

Relational Algebra is a theoretical procedural query language where queries take one or more relations as input and produce a new relation as output: 1. **Selection ($\sigma_C(R)$):** Filters rows satisfying predicate condition C (corresponds to SQL `WHERE`). 2. **Projection ($\pi_{A_1, A_2}(R)$):** Filters specific columns, eliminating duplicates (corresponds to SQL `SELECT DISTINCT`). 3. **Cartesian Product ($R \times S$):** Pairs every tuple in R with every tuple in S. 4. **Natural Join ($R \bowtie S$):** Pairs tuples having identical values across common attributes, projecting the common attribute once. 5. **Division ($R \div S$):** Returns tuples in R that are associated with *all* tuples in S (corresponds to queries containing 'FOR ALL', e.g., 'Find candidates who have completed all courses').

Query Optimization Heuristics: Modern query optimizers transform abstract SQL into relational algebra trees and apply algebraic rewrite rules: **pushing selections and projections down** as close to disk scans as possible to minimize intermediate relation sizes.

· 2. Key Formulas, Rules & Complexity Classes

1. **Division Operator Semantics:** $$R(A, B) \div S(B) = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$$ 2. **Selection Push-Down Optimization:** $$\sigma_C(R \bowtie S) \equiv (\sigma_C(R)) \bowtie S \quad \text{(if condition } C \text{ references only } R)$$

· 3. Step-by-Step Solved Examples

Example 3.1: Solving 'Find Students who Enrolled in ALL Courses' via Relational

- **Difficulty / Target Companies:** `Medium | GATE, Amazon, Oracle`
- **Problem Statement:** Given `Enrolled(student_id, course_id)` and `Course(course_id)`, express in Relational Algebra the students enrolled in every course.

Step-by-Step Walkthrough & Logic:

- Step 1: Identify all available courses: $\pi_{course_id}(Course)$.

- Step 2: Relational division of `Enrolled` by `Course` eliminates courses and retains student IDs associated with all course records.
- Step 3: Relational Algebra: $\pi_{student_id, course_id}(Enrolled) \div \pi_{course_id}(Course)$.

Complete Solved Code / Calculation:

```
-- SQL Equivalent of Relational Division:
SELECT student_id
FROM Enrolled
GROUP BY student_id
HAVING COUNT(DISTINCT course_id) = (SELECT COUNT(*) FROM Course);
```

- **Line-by-Line Breakdown & Complexity Analysis:** Group by with count equals total demonstrates the exact mathematical equivalence of Relational Division in SQL.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: Which relational algebra operator is non-commutative?

Options:

- A) Union
- B) Set Difference
- C) Natural Join
- D) Intersection

Answer: **B) Set Difference** **Detailed Solution & Reasoning:** Set Difference is not commutative: $R - S

e S - R$.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Google)

- **Interview Question:** _How do database query planners use relational algebra equivalence rules to optimize query execution plans?_
- **Model High-Scoring Answer:** Query planners parse SQL into an Abstract Syntax Tree of relational algebra operators. They apply commutative, associative, and distributive algebraic rewrite rules: 1) **Selection Push-Down:** Moving σ below joins drastically filters rows before expensive Cartesian cross-products occur. 2) **Projection Push-Down:** Dropping unused columns early minimizes I/O buffer footprint. 3) **Join Reordering:** Dynamically ordering joins so smaller intermediate relations are joined first (e.g., Hash Joins with smallest build tables).

- **Interviewer Follow-Up & Pro Tip:** ? Cite Selection Push-Down as the most important relational algebra optimization.

CHAPTER 4

Functional Dependencies & Higher Normal Forms

Key Principle / Exam Takeaway: Master Multivalued Dependencies (MVD), Fourth Normal Form (4NF), Join Dependencies (JD), and Fifth Normal Form (5NF / Project-Join Normal Form).

Core Curriculum Topics Covered in this Chapter:

- Review of 1NF, 2NF, 3NF, BCNF
- Multivalued Dependencies (MVD): Definition and Chase Test
- Fourth Normal Form (4NF): Eliminating independent multi-valued facts
- Join Dependencies (JD) and lossy join decomposition
- Fifth Normal Form (5NF / PJNF) and Domain-Key Normal Form (DKNF)

· 1. Core Concepts & In-Depth Explanation

While 1NF through BCNF address functional dependencies ($X \to Y$), higher normal forms resolve multi-valued redundancy:

- **Multivalued Dependency ($X \twoheadrightarrow Y$):** In relation $R(X, Y, Z)$, if for any given value of X, the set of values for Y is completely independent of the values for Z.
- **Fourth Normal Form (4NF):** A relation is in 4NF if it is in BCNF and for every non-trivial MVD $X \twoheadrightarrow Y$, X is a super key. Violating 4NF creates combinatorial Cartesian multiplication of rows (e.g., storing a Teacher's independent Phone Numbers and Subjects in the same table).
- **Fifth Normal Form (5NF / Project-Join NF):** A relation is in 5NF if it cannot be non-loss decomposed into smaller tables unless every join dependency is implied by candidate keys.

· 2. Key Formulas, Rules & Complexity Classes

1. **MVD Triviality:** $$X \twoheadrightarrow Y \text{ is trivial if } Y \subseteq X \text{ or } X \cup Y = R$$ 2. **4NF Rule:** $$\text{For every non-trivial } X \twoheadrightarrow Y, \quad X \text{ MUST be a Super Key!}$$

· 3. Step-by-Step Solved Examples

Example 4.1: Decomposing Relation to 4NF to Eliminate Combinatorial Duplication

- **Difficulty / Target Companies:** `Medium | Oracle, Amazon`
- **Problem Statement:** Relation `Lecturer(lec_id, phone_number, subject)` where phone numbers and subjects taught are independent multi-valued facts. Explain why it violates 4NF and decompose.

Step-by-Step Walkthrough & Logic:

- Step 1: If Lecturer 1 has 2 phones and teaches 3 subjects, the table requires $2 imes 3 = 6$ rows!
- Step 2: Non-trivial MVDs exist: `lec_id ->> phone_number` and `lec_id ->> subject`.

- Step 3: `lec_id` is not a super key (the only key is composite of all 3 columns). Violates 4NF.

- Step 4: Decompose into two independent 4NF relations: `LecPhones(lec_id, phone_number)` and `LecSubjects(lec_id, subject)`.

Complete Solved Code / Calculation:

```
-- 4NF Normalized Decomposition:
CREATE TABLE LecPhones (
    lec_id INT,
    phone_number VARCHAR(20),
    PRIMARY KEY (lec_id, phone_number)
);

CREATE TABLE LecSubjects (
    lec_id INT,
    subject VARCHAR(100),
    PRIMARY KEY (lec_id, subject)
);
```

- **Line-by-Line Breakdown & Complexity Analysis:** Eliminates combinatorial duplication: 2 phones + 3 subjects = 5 rows total instead of 6 multiplied rows.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: When does an update anomaly occur in a table violating 4NF?

Options:

- A) Only when deleting rows
- B) Adding a new phone number requires duplicating it across every subject the lecturer teaches

- C) When using indexes
- D) Never

Answer: **B) Adding a new phone number requires duplicating it across every subject the lecturer teaches** **Detailed Solution & Reasoning:** Independent multi-valued attributes multiply combinatorially, forcing repeated inserts.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Amazon)

- **Interview Question:** _What is Fifth Normal Form (5NF) and is it commonly enforced in practical commercial software design?_
- **Model High-Scoring Answer:** 5NF (Project-Join Normal Form) deals with cases where a table can be decomposed into three or more relations without loss of information, but cannot be decomposed into two tables without losing information. It is exceedingly rare in commercial software engineering: nearly all relational databases normalize up to 3NF or BCNF, occasionally 4NF. Normalizing beyond BCNF adds excessive join complexity that degrades read performance in high-throughput applications.

- **Interviewer Follow-Up & Pro Tip:** ? Acknowledge 5NF theoretically, but emphasize 3NF/BCNF as pragmatic industry standards.

CHAPTER 5

Storage Structures, File Organization & Indexing

Key Principle / Exam Takeaway: Master Fixed vs Variable length records, Clustered vs Secondary indexes, B-Trees vs B+ Trees, and Hash Indexing.

Core Curriculum Topics Covered in this Chapter:

- Physical file organization: Heap files, Sequential sorted files, Hashing
- Page layout: Slotted-page architecture for variable-length records and NULL bitmaps
- B-Trees vs B+ Trees: Internal node key storage vs leaf linked lists
- Clustered (Primary) vs Non-Clustered (Secondary) index pointer mechanics
- Hash Indexing: Static hashing, Extendible Hashing, and Linear Hashing

· 1. Core Concepts & In-Depth Explanation

Database storage engines manage data persistence across physical disk blocks:

- **Slotted-Page Architecture:** Standard layout for variable-length records. Page header contains a slot array growing downward storing `[offset, length]` pointers; actual tuple data grows upward from the page bottom. Enables tuple defragmentation within the page without invalidating external `RecordID (page_id, slot_id)`.
- **B+ Tree vs B-Tree:** In a B-Tree, data records or pointers are stored in all nodes (internal and leaf). In a **B+ Tree**, internal nodes store ONLY navigation guide keys and child page pointers, while ALL actual data pointers reside in leaf nodes. The leaves are linked together in a bidirectional doubly linked list.
- Advantages of B+ Tree: 1) Higher fan-out because internal nodes are compact (lower tree height). 2) Extremely fast range scans by traversing the leaf linked list without navigating back up the tree.

· 2. Key Formulas, Rules & Complexity Classes

1. **B+ Tree Fan-out:** $$\text{Fan-out } M = \frac{\text{Page Size}}{\text{Key Size} + \text{Pointer Size}} \approx \frac{4096}{8 + 8} = 256$$ 2. **Search Cost:** $$\text{Disk Reads} = \lceil \log_M(N) \rceil \le 3-4 \text{ reads for } 10^8 \text{ records}$$

· 3. Step-by-Step Solved Examples

Example 5.1: Computing Slotted Page Record Offset and Free Space

- **Difficulty / Target Companies:** `Medium | Amazon, Microsoft`
- **Problem Statement:** In a 4096-byte page with a 64-byte header, insert two 200-byte tuples. Show slot array and free space boundary.

Step-by-Step Walkthrough & Logic:

- Step 1: Page size = 4096 bytes. Header occupies bytes 0-63.
- Step 2: Slot 0 (4 bytes: offset + length) added at byte 64. Tuple 1 placed at bottom: bytes [3896 to 4095]. Slot 0 points to 3896.
- Step 3: Slot 1 added at byte 68. Tuple 2 placed at bytes [3696 to 3895]. Slot 1 points to 3696.

- Step 4: Free space window is between byte 72 (end of slot array) and byte 3696 (bottom of data).

Complete Solved Code / Calculation:

```
// Slotted Page Layout:
// [ Header (0-63) ][ Slot 0 (64-67) ][ Slot 1 (68-71) ][ ... Free Space ... ][ Tuple 2 (3696-3
```

- **Line-by-Line Breakdown & Complexity Analysis:** Enables $O(1)$ tuple addressing via RecordID `(page_num, slot_num)`.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: Why are B+ Trees preferred over Hash Indexes for general-purpose database query processing?

Options:

- A) B+ Trees are faster for exact matches
- B) B+ Trees support range queries and sorted scans (WHERE age BETWEEN 20 AND 30); Hash Indexes only support exact equality
- C) Hash indexes use more memory
- D) B+ Trees eliminate disk I/O

Answer: **B) B+ Trees support range queries and sorted scans (WHERE age BETWEEN 20 AND 30); Hash Indexes only support exact equality** **Detailed Solution & Reasoning:** Hash functions scatter data pseudo-randomly, destroying range ordering; B+ Tree leaves maintain sorted linked lists.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Amazon)

- **Interview Question:** _What is the difference between a Clustered Index and a Non-Clustered Index?_

- **Model High-Scoring Answer:** A Clustered Index defines the actual physical on-disk storage order of the data rows. Consequently, a table can have only ONE clustered index. In a clustered B+ tree, the leaf pages *are* the data pages. A Non-Clustered (Secondary) Index is stored in a separate B+ tree structure. Its leaf pages contain index keys and pointers (such as the Clustered Key or RowID) to locate the actual data row in the clustered table. Secondary index queries that select un-indexed columns incur a secondary lookup overhead ('Bookmark Lookup / Index Jump').

- **Interviewer Follow-Up & Pro Tip:** ? Mention Bookmark Lookup cost and Covering Indexes to avoid it.

CHAPTER 6

Transaction Processing, Conflict Serializability & Strict 2PL

Key Principle / Exam Takeaway: Master Conflict Serializability, Precedence Graphs (Serialization Graphs), Two-Phase Locking (2PL), and Strict 2PL for cascading abort prevention.

Core Curriculum Topics Covered in this Chapter:

- Transaction states: Active, Partially Committed, Committed, Failed, Aborted
- Conflicting operations: Read-Write, Write-Read, Write-Write on identical items
- Conflict Serializability testing via Precedence Graphs (Cycle detection)
- Two-Phase Locking (2PL): Growing Phase vs Shrinking Phase
- Strict 2PL and Rigorous 2PL: Holding exclusive locks until COMMIT/ABORT

· 1. Core Concepts & In-Depth Explanation

Concurrent execution of database transactions requires ensuring serializable consistency without sacrificing throughput.

Conflict Serializability: Two operations conflict if they belong to different transactions, access the identical data item, and at least one is a `write`. A concurrent schedule is **Conflict Serializable** if it is conflict-equivalent to some serial schedule.

- **Precedence Graph (Serialization Graph):** Directed graph where nodes are transactions. A directed edge $T_i \to T_j$ exists if an operation in T_i conflicts with and executes before an operation in T_j.
- **Theorem:** A schedule is conflict serializable if and only if its precedence graph contains **NO CYCLES**.

Two-Phase Locking (2PL): Guarantees conflict serializability: 1. **Growing Phase:** Transaction may acquire locks, but cannot release any lock. 2. **Shrinking Phase:** Transaction may release locks, but cannot acquire any new lock.

Strict 2PL: Prevents **Cascading Aborts** by holding all Exclusive (`X`) write locks until the transaction commits or aborts.

· 2. Key Formulas, Rules & Complexity Classes

1. **Conflict Equivalence Rule:** $$\text{Cycle in Precedence Graph} \implies \text{NOT Conflict Serializable!}$$ 2. **Strict 2PL Lock Release:** $$\text{Hold all } X \text{-locks until COMMIT} \implies \text{Cascadeless schedule}$$

· 3. Step-by-Step Solved Examples

Example 6.1: Testing Conflict Serializability using Precedence Graph

- **Difficulty / Target Companies:** `Medium | GATE, Amazon, Oracle`
- **Problem Statement:** Schedule $S: R_1(A); W_2(A); R_1(B); W_2(B); R_2(C); W_1(C)$. Is S conflict serializable?

Step-by-Step Walkthrough & Logic:

- Step 1: Conflicting pairs on A: $R_1(A)$ before $W_2(A) \implies$ Edge $T_1 o T_2$.
- Step 2: Conflicting pairs on B: $R_1(B)$ before $W_2(B) \implies$ Edge $T_1 o T_2$.
- Step 3: Conflicting pairs on C: $R_2(C)$ before $W_1(C) \implies$ Edge $T_2 o T_1$.
- Step 4: Check graph: Edges are $T_1 o T_2$ and $T_2 o T_1$. A CYCLE exists ($T_1 \leftrightarrows T_2$)!
- Step 5: Schedule S is NOT conflict serializable.

Complete Solved Code / Calculation:

```
// Precedence Graph:
// T1 ----> T2 (due to A and B)
// T2 ----> T1 (due to C)
// Cycle detected! Non-serializable schedule!
```

- **Line-by-Line Breakdown & Complexity Analysis:** Illustrates why interleaving conflicting reads and writes in reverse orders destroys serializability.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: What is the primary benefit of Strict Two-Phase Locking (Strict 2PL) over basic 2PL?

Options:

- A) Prevents Deadlocks
- B) Prevents Cascading Aborts (Cascading Rollbacks)
- C) Increases concurrency
- D) Eliminates shared locks

Answer: **B) Prevents Cascading Aborts (Cascading Rollbacks)** **Detailed Solution & Reasoning:** By holding exclusive locks until final commit, uncommitted dirty writes are never exposed to other transactions.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Google)

- **Interview Question:** _Does Two-Phase Locking (2PL) prevent Deadlocks?_
- **Model High-Scoring Answer:** No. While 2PL guarantees Conflict Serializability, it does NOT prevent deadlocks. For example, T_1 holds a lock on A and requests a lock on B, while T_2 holds a lock on B and requests a lock on A. Both transactions are in their growing phase and freeze in a deadlock (Circular Wait). Systems must use deadlock detection (Wait-For Graph cycles) or avoidance (Wait-Die / Wound-Wait schemes based on timestamps).
- **Interviewer Follow-Up & Pro Tip:** ? Decouple serializability from deadlock freedom.

CHAPTER 7

Distributed Databases, Replication & CAP Theorem

Key Principle / Exam Takeaway: Master CAP Theorem (Consistency, Availability, Partition Tolerance), PACELC, Master-Slave vs Multi-Master replication, and Sharding.

Core Curriculum Topics Covered in this Chapter:

- Distributed data systems: Scaling Up (Vertical) vs Scaling Out (Horizontal)
- CAP Theorem: In the presence of a Network Partition (P), choose Consistency (C) or Availability (A)
- PACELC Theorem: Extending CAP for normal non-partitioned operation (Latency vs Consistency)
- Replication strategies: Synchronous vs Asynchronous replication, Read Replicas
- Horizontal Partitioning (Sharding): Range-based, Hash-based, and Consistent Hashing

· 1. Core Concepts & In-Depth Explanation

Modern internet architectures distribute data across multiple physical servers:

- **CAP Theorem (Brewer):** Any distributed data store can guarantee at most two of three properties:
- **Consistency (Linearizability):** Every read receives the most recent write or an error.
- **Availability:** Every non-failing node returns a non-error response (without guarantee that it contains the most recent write).
- **Partition Tolerance:** The system continues operating despite arbitrary dropped or delayed network messages between nodes.

Because physical networks inevitably experience partitions (P), real systems must choose between CP (Consistency over Availability) or AP (Availability over Consistency).

Sharding & Consistent Hashing: Distributes huge datasets across nodes. Simple modulo hashing (`hash(key) % N`) causes massive data migration when nodes are added or removed. **Consistent Hashing** maps both servers and data keys onto a circular hash ring (0 to $2^{32}-1$), ensuring that adding or removing a node relocates only K/N keys on average.

· 2. Key Formulas, Rules & Complexity Classes

1. **Consistent Hashing Rebalance Bound:** $$\text{Keys Rebalanced} = \frac{K}{N} \quad \text{(vs } \approx 100\% \text{ in simple modulo hashing)}$$

· 3. Step-by-Step Solved Examples

Example 7.1: Applying Consistent Hashing on a Distributed Key-Value Store

- **Difficulty / Target Companies:** `Medium | Amazon, Uber, Netflix`
- **Problem Statement:** Explain how virtual nodes in Consistent Hashing prevent hot-spots and uneven data distribution.

Step-by-Step Walkthrough & Logic:

- Step 1: Physical nodes are mapped to multiple random positions on the hash ring using virtual nodes (e.g., Node1#1, Node1#2).
- Step 2: Keys are hashed onto the ring and stored in the first clockwise virtual node.
- Step 3: Virtual nodes distribute key ownership evenly across the physical hardware, eliminating load skew.

Complete Solved Code / Calculation:

```
// Consistent Hashing Ring:
// Ring: 0 ----------------------------------> 2^32 - 1
// Key 'user_101' hashed to pos 45,000 -> Stored on Server B (pos 50,000)
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees smooth load distribution across heterogeneous physical clusters.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: Under the CAP Theorem, why is CA (Consistency + Availability) impossible in a real-world distributed network?

Options:

- A) Because hardware is cheap
- B) Because physical network cables and routers inevitably drop or delay packets (Partition Tolerance is mandatory)
- C) Only works on single servers
- D) Because SQL is ACID

Answer: **B) Because physical network cables and routers inevitably drop or delay packets (Partition Tolerance is mandatory)**
Detailed Solution & Reasoning: Network partitions are unavoidable physical realities; when a partition occurs, a system must choose to either reject reads/writes (CP) or serve potentially stale data (AP).

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Amazon)

- **Interview Question:** _What is Eventual Consistency and how does it differ from Strong Consistency?_
- **Model High-Scoring Answer:** In Strong Consistency (Linearizability), once a write commits, all subsequent reads across all cluster nodes return that updated value immediately. In Eventual Consistency, replicas synchronize asynchronously: if no further updates are made to a key, all replicas will eventually converge to identical values. Reads made immediately following a write may return stale data. Systems choose Eventual Consistency (e.g., Amazon DynamoDB, Cassandra) to achieve high availability and low write latencies.
- **Interviewer Follow-Up & Pro Tip:** ? Cite the latency and availability trade-off.

CHAPTER 8

High-Level System Design for Campus Placements

Key Principle / Exam Takeaway: Master high-level architecture: URL Shortener (TinyURL), Distributed Rate Limiter (Token Bucket), and Cache Aside pattern.

Core Curriculum Topics Covered in this Chapter:

- System Design Interview Framework: Requirements, Back-of-the-envelope estimation, High-Level Architecture, Deep Dives
- Designing a URL Shortener (TinyURL): Base62 encoding, unique ID generation, Redis caching
- Rate Limiting Algorithms: Token Bucket, Leaky Bucket, Sliding Window Counter
- Caching Strategies: Cache-Aside, Write-Through, Write-Behind, Cache Invalidation
- CDN, Load Balancers (Layer 4 vs Layer 7), and Database Read-Replicas

· 1. Core Concepts & In-Depth Explanation

High-Level System Design evaluates architectural scalability, trade-off analysis, and component orchestration.

Core Building Blocks: 1. **Load Balancer (Nginx/HAProxy):** Distributes incoming traffic across stateless application instances (Round Robin, Least Connections). 2. **Caching (Redis/Memcached):** In-memory cache deployed in front of the database. Common pattern is **Cache-Aside**: application reads from cache; on miss, reads from DB and writes to cache. 3. **Database Scaling:** Read Replicas for read-heavy workloads; Horizontal Sharding for write scaling. 4. **Rate Limiting:** Protects backend services from traffic spikes and DoS using algorithms like **Token Bucket** (tokens added at steady rate; requests consume tokens; if empty, drop request with HTTP 429 Too Many Requests).

· 2. Key Formulas, Rules & Complexity Classes

1. **Base62 URL Shortener Key Capacity:** $$62^7 = 3.52 \times 10^{12} \text{ unique short URLs using 7 alphanumeric characters [0-9, a-z, A-Z]}$$ 2. **Token Bucket Rate Check:** $$\text{tokens} = \min(\text{capacity}, \text{tokens} + \text{elapsed} \times \text{refill_rate})$$

· 3. Step-by-Step Solved Examples

Example 8.1: Designing TinyURL: Base62 Encoding vs MD5 Hashing

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Uber`
- **Problem Statement:** Design a URL shortening service handling 100 million new URLs per month.

Step-by-Step Walkthrough & Logic:

- Step 1: Use a 64-bit distributed auto-increment ID generator (e.g., Twitter Snowflake).
- Step 2: Convert 64-bit integer ID to Base62 string using characters [0-9a-zA-Z].
- Step 3: Store mapping in database: `(id, short_key, original_url, created_at)`.
- Step 4: Cache top 20% hot URLs in Redis with LRU eviction.

Complete Solved Code / Calculation:

```
def encode_base62(num: int) -> str:
    CHARS = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
    if num == 0:
        return CHARS[0]
    res = []
    while num > 0:
        res.append(CHARS[num % 62])
        num //= 62
    return "".join(reversed(res))

# Example: ID 125,300 -> 'wE4' (Deterministic, collision-free!)
```

- **Line-by-Line Breakdown & Complexity Analysis:** Base62 conversion of a unique integer is 100% collision-free, completely avoiding MD5 hash collisions.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: In the Cache-Aside pattern, what is the correct sequence when updating data?

Options:

- A) Update cache, then update database
- B) Update database first, then invalidate (delete) the entry in the cache
- C) Never delete from cache
- D) Write to disk only

Answer: **B) Update database first, then invalidate (delete) the entry in the cache** **Detailed Solution & Reasoning:** Updating DB then invalidating cache minimizes race conditions and prevents stale cache data.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Amazon)

- **Interview Question:** _How do you implement a distributed rate limiter across multiple web server nodes?_

- **Model High-Scoring Answer:** Store token bucket counters centrally in **Redis** using atomic Lua scripts or Redis transactions. When a request arrives at any web server node, it executes an atomic Lua script in Redis that: 1) Fetches the token count and timestamp for `user_id`. 2) Adds newly refilled tokens based on elapsed time. 3) If tokens >= 1, decrements 1 token and returns 1 (allow). 4) If tokens < 1, returns 0 (reject with HTTP 429). Storing state in Redis ensures rate limits are globally enforced regardless of which load-balanced server handles the request.

- **Interviewer Follow-Up & Pro Tip:** ? Cite Redis atomic Lua scripts to prevent race conditions.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.